DevOps Engineer Take-Home Assignment

October 2026

Overview

Plan for about **4 hours**. We'd rather see a smaller piece done well, with your assumptions written down, than every box ticked. If you run out of time, write a few lines on what you'd do next and stop there.

The brief tells you what to build, not how to build it. The design decisions are yours, and you don't need to justify each one in writing. We'll talk through them in the review session.

You'll produce three things in one Git repo:

1. **Terraform** for a multi-AZ web tier (Part 1)
2. **A GitHub Actions workflow** with a short promotion and rollback write-up (Part 2)
3. **A design doc**, 2–3 pages, covering migration, DR, observability, edge and security (Part 3)

After you submit, we'll book a 90-minute review session. You walk us through your work, then we ask a few more questions about your past experience. That part needs no prep.

Nothing needs to run against a real AWS account. You don't need to apply anything or spend money. `terraform validate` and a clean `plan` against mocked or placeholder values are enough.

AI tools are allowed and expected. Use whatever you normally use. Add a short note in the README on where you used AI and how. In the review session we'll go through the code line by line, so make sure you can explain and defend every choice.

The scenario

Maple Online Academy is a made-up online high school with about 15,000 active students. Its whole platform runs on one server rented from a small VPS provider in Edmonton, Alberta, and the business wants it on AWS without downtime during exam periods.

What runs today:

- **LMS:** a PHP 8.2 monolith on Apache. It reads and writes a MySQL 8 database (~40 GB) on the same host.
- **Store:** a WordPress + WooCommerce site on the same host. It handles course purchases and has its own MySQL database.
- **Background jobs:** about 25 cron jobs (grade sync, email digests, report generation). The crontab lives only on the server, not in Git.
- **Files:** course content was recently moved to S3. Student uploads (assignments, profile photos) are still on local disk, about 200 GB.
- **Edge:** Cloudflare handles DNS and proxies traffic. Only default settings are on, and the origin IP is reachable directly.
- **Deploys:** a developer SSHes in, runs `git pull`, and clears the cache. There is no staging environment.
- **Backups:** a nightly `mysqldump` to the same disk, copied to S3 "most nights." No one has tested a restore.
- **Monitoring:** an external uptime ping and Sentry for PHP errors.

What the business cares about:

- **Exam windows.** Proctored final exams run on weekday business hours, with peaks at about 3× normal traffic. An outage during an exam means rescheduling students and supervisors, and sometimes refunds.
- **Purchases.** If checkout is down, revenue is lost directly.
- **Academic records.** Grades and transcripts are legal records. Losing them is not acceptable.
- **Ransomware.** Leadership worries a single compromised server could take out everything, backups included.
- **Data residency.** Student data is personal information under PIPEDA. Assume it should stay in Canada (`ca-central-1`).

AWS is already set up: a multi-account organization (Control Tower) with IAM Identity Center for human access, and separate `dev` and `prod` workload accounts. GitHub Actions can assume roles in those accounts through OIDC.

A separate security company will run a pen test once the platform is on AWS. You'll work through its findings with them.

Where the brief leaves something out or is unclear, make a reasonable assumption, write it down and keep going.

Part 1 — Infrastructure as Code (~1.5 h)

Write Terraform for the LMS's target home on AWS in `ca-central-1`, with a `dev` and a `prod` environment. Make the choices you'd make for a real production system.

It should include:

- Networking across at least two AZs
- A load-balanced compute tier for the PHP app
- A managed MySQL 8 database
- Storage for student uploads
- IAM and secrets wiring so the app can reach the database and the storage

Expectations:

- `terraform fmt` and `terraform validate` pass, and a `plan` succeeds with placeholder values. Don't apply.
- Remote state configuration is shown, even if it points at a bucket that doesn't exist.
- How you structure the code across environments is up to you. Terragrunt is welcome but not required.
- You don't need to cover the WordPress store, cron jobs or Cloudflare in code.

In the README, list what you left out.

Part 2 — CI/CD (~1 h)

Replace "SSH in and `git pull`" with a GitHub Actions pipeline that ships the PHP app to the infrastructure from Part 1. The workflow doesn't have to run, but it should be realistic enough that we could finish it in a day.

Hand in:

- **Workflow file(s)** that take a change from merge to running in `prod`, by way of `dev`.
- **The trust policy** for the AWS role that GitHub Actions assumes.
- **A half-page note** on how releases and rollbacks work in your setup.

Assume the app has a `composer.json` and a small PHPUnit suite.

Part 3 — Design doc (~1.5 h)

Write 2–3 pages, in Markdown, for an engineering lead who has to approve the plan. Cover the topics below. How deep you go on each is your call.

1. **Migration**: getting from the single server to AWS, including the store, the cron jobs and the uploads.
2. **Backups and DR**: including the RTO and RPO targets you'd commit to.
3. **Observability and alerting.**
4. **Cloudflare edge.**
5. **Access and secrets.**

End with a short list of what you'd tackle in your first 90 days, in priority order, with a rough time estimate for each.

Review session (90 min, no prep)

- **~60 min: walkthrough.** You present your repo and design doc. We'll dig into the decisions you made, including the ones you didn't write down.
- **~10 min: your experience.** Additional questions and discussion based on your previous experience.
- **~10 min: your questions.**

The remaining 10 minutes are slack, so nothing gets rushed.

Submitting and how we evaluate

Send us a link to a **private GitHub repo** (we'll tell you who to invite) or a zip file, within **the agreed-upon timeline**. The README should cover how the repo is laid out, your assumptions, what you'd do next, and your AI-usage note.

We'll look at:

- **Correctness.** The code would work with small fixes, not big rewrites.
- **Judgement.** What you chose to build, protect and leave out.
- **Depth.** How well you can explain the trade-offs behind your choices from hands-on experience. Most of this comes out in the review session.
- **Clear writing.** A lead could approve the plan from your doc alone.